

Experimental Analysis of Sorting Algorithms

An Empirical Study Across Sizes and Input Shapes

Maria Maiorescu

Faculty of Informatics
West University of Timișoara

May 21, 2026

Goal: compare six classical sorting algorithms using different array sizes and input structures, combining theoretical complexity analysis with practical benchmarking implemented in C.

Sorting is fundamental in computer science:

- Databases
- Search engines
- Operating systems

Theory focuses on asymptotic complexity, but practical performance also depends on **implementation details** and **input structure**.

Why benchmarking matters

Asymptotic bounds hide constants, cache effects, and the shape of real data. An algorithm that wins on paper may lose on real input.

Consequence

Practical benchmarking is *essential* alongside theoretical analysis.

Related Work

- **Donald E. Knuth (1998)** — *The Art of Computer Programming, Vol. 3*: foundational reference for sorting theory and complexity proofs.
- **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein (2022)** — *Introduction to Algorithms (CLRS)*: standard source for algorithm design and the $\Omega(n \log n)$ lower bound.
- **Nicolas Auger, Vincent Jugé, Cyril Nicaud, Carine Pivoteau (2019)** — proved TimSort runs in $\mathcal{O}(n \log n)$ worst case; TimSort exploits adaptive behaviour of Insertion Sort on nearly-sorted data.
- **Robert Sedgwick & Jon Bentley (2002)** — argued that well-engineered Quick Sort achieves near-optimal average performance.

What makes this project different

Unlike many previous studies, this project compares **six algorithms** under **identical testing conditions** and **multiple input structures**.

Algorithms Used

- **Bubble Sort** – $\mathcal{O}(n^2)$, early-exit flag
- **Insertion Sort** – $\mathcal{O}(n^2)$; drops to $\mathcal{O}(n)$ on nearly-sorted input
- **Selection Sort** – $\mathcal{O}(n^2)$, always
- **Merge Sort** – $\mathcal{O}(n \log n)$, guaranteed
- **Quick Sort** – $\mathcal{O}(n \log n)$ average; median-of-three pivot
- **Radix Sort (LSD)** – $\mathcal{O}(d \cdot n)$, bypasses the comparison lower bound

Algorithm	Best	Avg	Worst	Space
Bubble	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Selection	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Merge	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n)$
Quick	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n \lg n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\lg n)$
Radix (LSD)	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(n+d)$

Key takeaway: the quadratic algorithms become inefficient on large arrays; Merge Sort and Quick Sort scale much better; Radix Sort achieved the fastest performance in most large tests.

Benchmarking framework implemented entirely in C:

- ① **Data generators** — fill array with the requested shape
- ② **Sorting routines** — six in-place implementations
- ③ **Timing layer** — wrappers around `clock()`
- ④ **Benchmarking loop** — repeats runs, writes `results.csv`

Timing formula:

$$t = \frac{t_{\text{end}} - t_{\text{start}}}{\text{CLOCKS_PER_SEC}} \times 1000 \text{ ms}$$

Input shapes tested:

- **Random** arrays
- **Reversed** arrays
- **Partially sorted** at five levels:
PS 10%, PS 30%, PS 50%, PS 70%, PS 90%

Averaging strategy

Small-array tests ($n \leq 100$) are averaged over 10^6 runs to reduce measurement noise.

Experimental Setup

Compiler and hardware

Personal laptop, Unix-like OS

```
gcc -O2 -std=c99
```

Array sizes tested:

$n \in$

{10, 50, 100, 1 000, 10 000, 100 000, 1 000 000}

Different input structures were used to observe how performance changes as the input size grows.

Repetitions per measurement:

Size n	Runs averaged
10, 50, 100	1,000,000
1,000	100
10,000	10
100,000 and above	1

Output: results.csv

Columns: Elements, Runs, Algorithm, Variant, Elapsed time(ms)

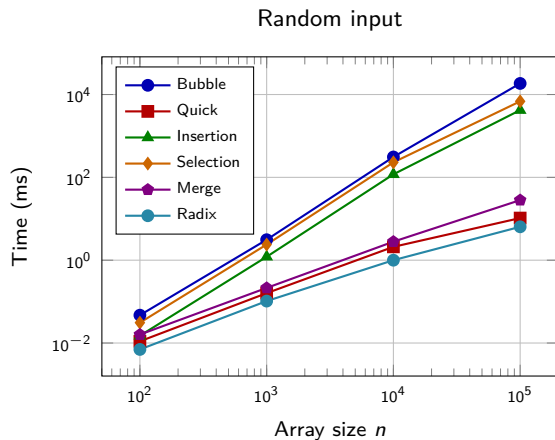
Results: Small and Medium Arrays

$n = 100$ — avg. over 10^6 runs (ms)

Algorithm	Rev.	Rand.	PS10	PS50	PS90
Bubble	0.037	0.047	0.027	0.017	0.013
Quick	0.036	0.011	0.015	0.032	0.037
Insertion	0.025	0.015	0.005	0.002	0.001
Selection	0.023	0.031	0.026	0.026	0.026
Merge	0.010	0.016	0.012	0.010	0.010
Radix	0.005	0.007	0.005	0.005	0.005

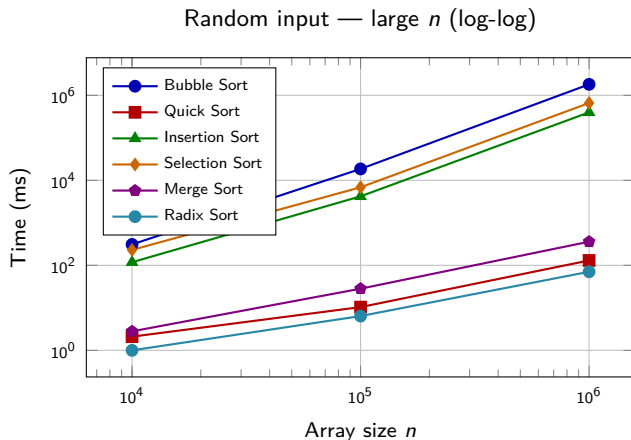
$n = 1,000$ — avg. over 100 runs (ms)

Algorithm	Rev.	Rand.	PS10	PS50	PS90
Bubble	3.711	3.120	2.537	2.264	2.169
Quick	3.215	0.160	0.217	0.622	1.052
Insertion	2.385	1.209	0.292	0.069	0.042
Selection	2.093	2.388	2.347	2.345	2.363
Merge	0.124	0.214	0.150	0.135	0.130
Radix	0.066	0.103	0.087	0.065	0.067



Results: Large Arrays

n	Algo.	Rev.	Rand.	PS90
10^4	Bubble	361.3	310.3	230.0
	Quick	312.1	2.1	10.6
	Insertion	235.5	118.4	3.7
	Merge	1.5	2.8	1.7
	Radix	0.9	1.0	0.9
10^5	Bubble	13 228	18 521	7 640
	Quick	12 726	10.4	72.1
	Insertion	8 343	4 199	122.1
	Merge	20.6	28.2	21.4
	Radix	5.9	6.4	5.2
10^6	Bubble	$\sim 10^6$ ms (off chart)		
	Quick	114.6	130.2	148.1
	Insertion	812k	397k	11 542
	Merge	232.7	359.7	227.0
	Radix	63.6	70.5	61.9



Analysis and Problems Encountered

Insertion Sort on nearly-sorted input

On PS 90% at $n = 10^6$: **11 542 ms** vs.
396 592 ms on random data — a **34×** speedup.
This is exactly the property that TimSort exploits.

Quick Sort on structured inputs

Reversed $n = 10,000$: **312.1 ms** — comparable to Bubble Sort.

Even median-of-three produces unbalanced partitions on arithmetic sequences.

Problem: Stack overflow

The recursive Quick Sort implementation caused a **segmentation fault** (stack overflow) on a partially-sorted $n = 10^6$ array because of deep recursion.

After adding the **median-of-three pivot** strategy the algorithm became much more stable.

Lesson

Practical implementations may behave very differently from theoretical expectations.

Summary:

- Compared **six classical sorting algorithms** across multiple array sizes and input structures.
- Confirmed the **limitations of quadratic algorithms** (unusable past $n \approx 10,000$).
- Highlighted the **efficiency of Merge Sort and Radix Sort** at scale.

Future work:

- Test **additional data types** (floating-point, strings).
- Compare against **standard library sorts** (qsort, Python Timsort).
- Implement **iterative Quick Sort** to eliminate stack overflow.
- Profile **cache behaviour** with Cachegrind.

Thank you for your attention!